



Compiladores

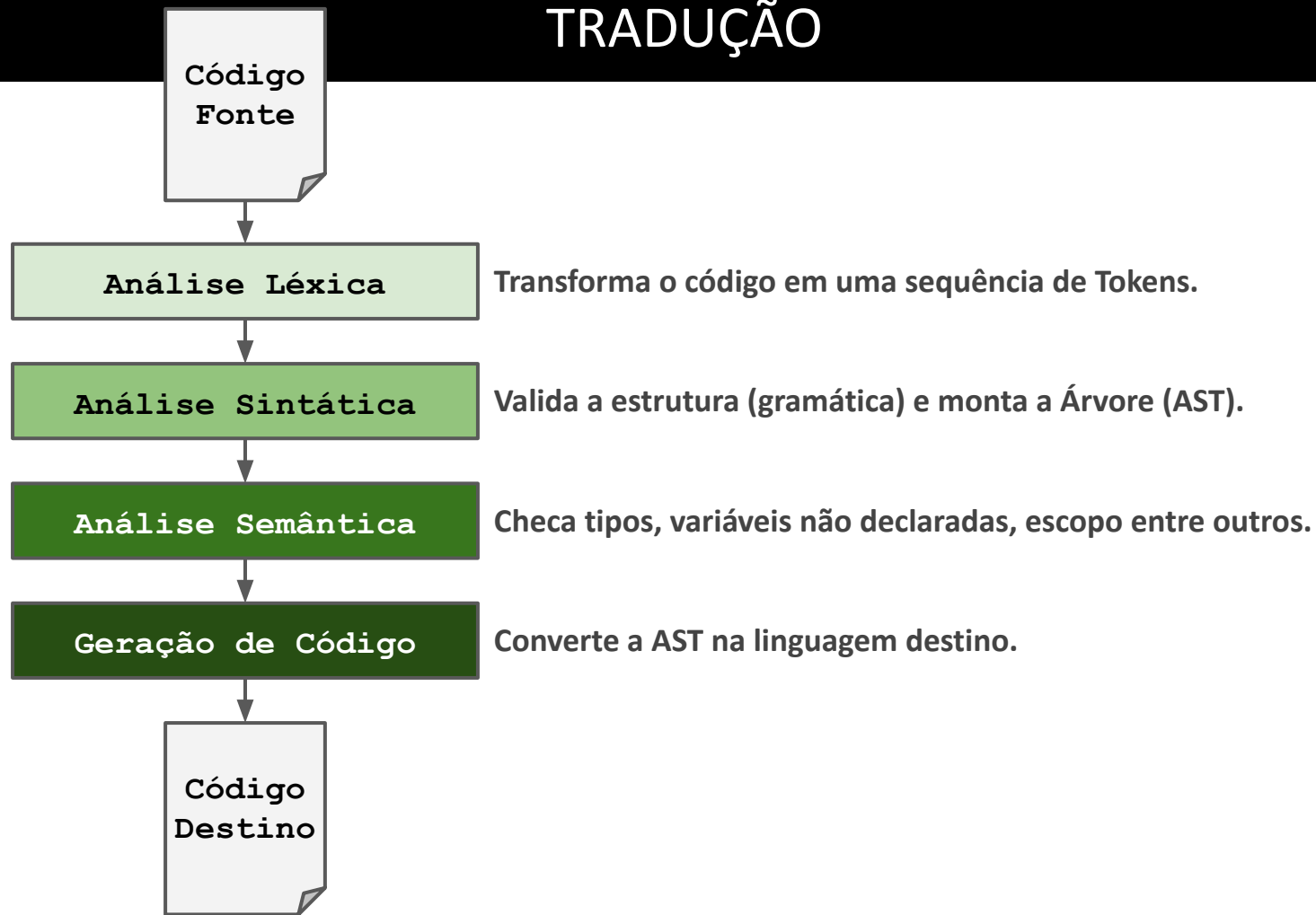
Material 5

AST

“Árvore Sintática Abstrata.”

Prof. Roger Rocha Ferreira

TRADUÇÃO



Erros nas Linguagens (Reais)

Léxico

Ocorre quando usamos uma palavra inexistente na língua ou com grafia incorreta.

*“O menino estava **tristoso** com a notícia.”*

Sintático

Acontece quando a estrutura da frase não segue as regras da gramática.

*“Eu **fomos** ao mercado ontem.”*

Semântico

Surge quando a frase está gramaticalmente correta, mas o sentido não faz sentido ou é inadequado.

*“O cachorro estava **lendo** o jornal.”*

Erros nas Pseudo-Linguagem de Computação

Léxico

São erros relacionados ao uso de símbolos ou palavras que não pertencem ao vocabulário da linguagem.

python

```
prnt("Olá mundo")
```

java

```
int número# = 10;
```

c

```
floatt x = 5.0;
```

Erros nas Pseudo-Linguagem de Computação

Sintático

São erros na forma como o código é escrito, violando as regras gramaticais da linguagem.

python : falta os dois pontos

```
if x > 10
    print("Maior que 10")
```

c : parêntese aberto incorretamente

```
int main( {
    return 0;
}
```

java : falta ponto e vírgula

```
public class Test {
    public static void
main(String[] args) {

    System.out.println("Olá
    mundo")

}
```

Erros nas Pseudo-Linguagem de Computação

Semântico

O código é sintaticamente válido, mas não faz o que o programador pretendia.

python : erro lógico: soma string com número

```
x = "10"  
y = x + 5
```

java : divisão por zero: sintaticamente correto, mas sem sentido

```
int a = 10;  
int b = 0;  
int c = a / b;
```

c : usa '=' (atribuição) em vez de '==' (comparação)

```
int idade = 20;  
if (idade = 18) {  
    printf("Maior de  
idade");  
}
```

Análise Semântica

Em tempo de Compilação

Java

C

VisualBasic

Em tempo de Interpretação

Python

JavaScript

MySQL (Linguagem de Consulta)

Sem Semântica

HTML (Linguagem de Marcação)

Assembly



BNF (Ajustado)

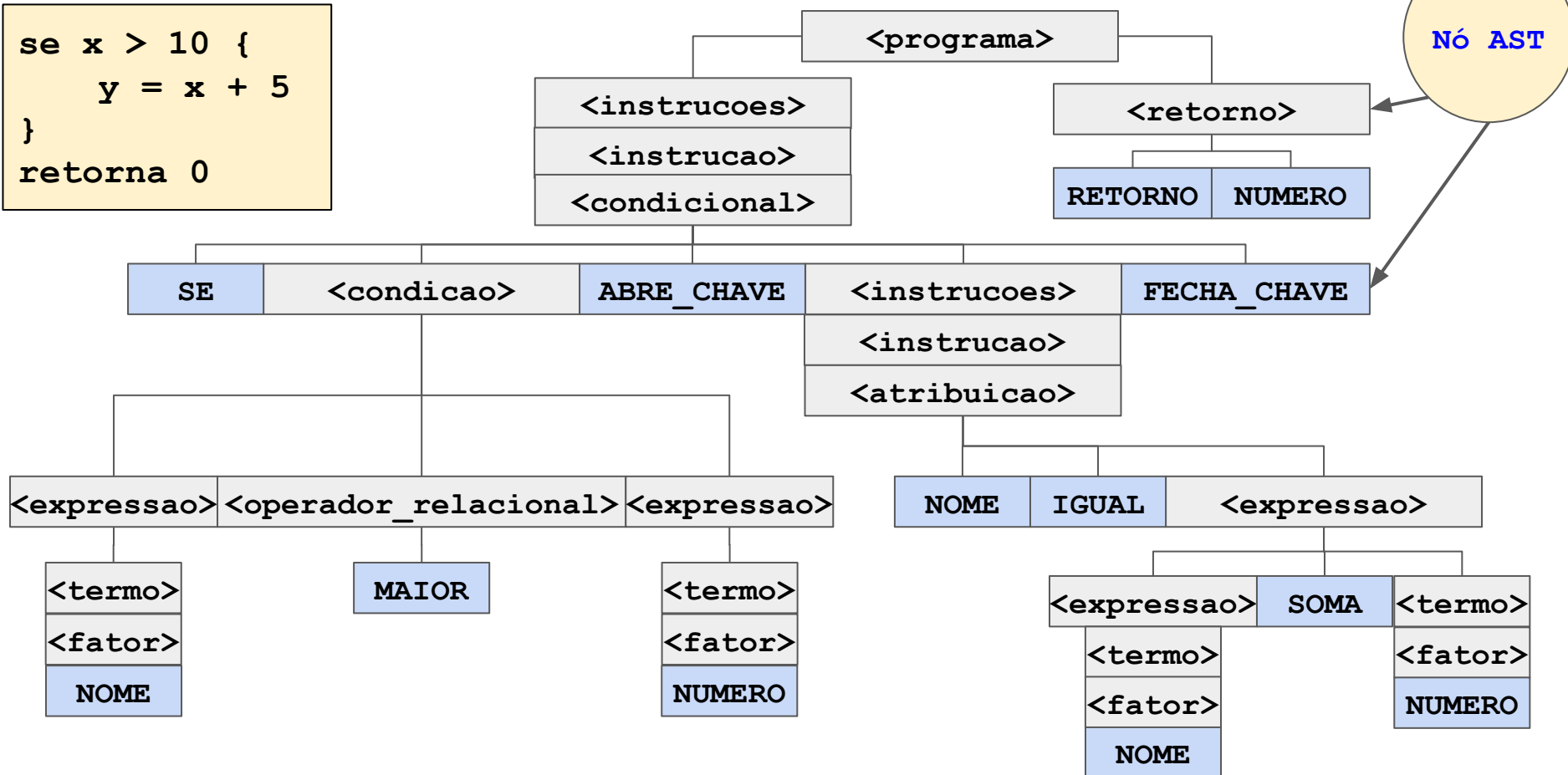
```
<programa> ::= <instrucoes> <retorno>
<instrucoes> ::= <instrucao> | <instrucao> <instrucoes>
<instrucao> ::= <atribuicao>
                | <saida>
                | <entrada>
                | <condicional>
                | <loop>
<atribuicao> ::= NOME IGUAL <expressao>
<saida> ::= ESCREVA ABRE_PAR <expressao> FECHA_PAR
<entrada> ::= NOME IGUAL LEIA ABRE_PAR TEXTO FECHA_PAR
<condicional> ::= SE <condicao> ABRE_CHAVE <instrucoes> FECHA_CHAVE
                | SE <condicao> ABRE_CHAVE <instrucoes> FECHA_CHAVE SENAO ABRE_CHAVE
<instrucoes> FECHA_CHAVE
<loop> ::= ENQUANTO <condicao> ABRE_CHAVE <instrucoes> FECHA_CHAVE
<condicao> ::= <expressao> <operador_relacional> <expressao>
              | NEGACAO <condicao>
<operador_relacional> ::= IGUALDADE | MENOR | MAIOR
<expressao> ::= <termo>
               | <expressao> SOMA <termo>
               | <expressao> SUB <termo>
```

```
<termo> ::= <fator>
           | <termo> MUL <fator>
           | <termo> DIV <fator>
<fator> ::= NUMERO
           | NOME
           | ABRE_PAR <expressao> FECHA_PAR
<retorno> ::= RETORNO NUMERO
```




AST - Árvore Sintática Abstrata

```
se x > 10 {  
    y = x + 5  
}  
retorna 0
```



Nó da AST - Árvore Sintática Abstrata

```
class NoAST:

    def __init__(self, tipo, valor=None, filhos=None):
        self.tipo = tipo
        self.valor = valor
        self.filhos = filhos if filhos is not None else []

    def __repr__(self, level=0):
        ret = "  " * level + f"[{self.tipo}"
        if self.valor is not None:
            ret += f": {self.valor}"
        ret += "]\n"
        for filho in self.filhos:
            ret += filho.__repr__(level + 1)
        return ret
```

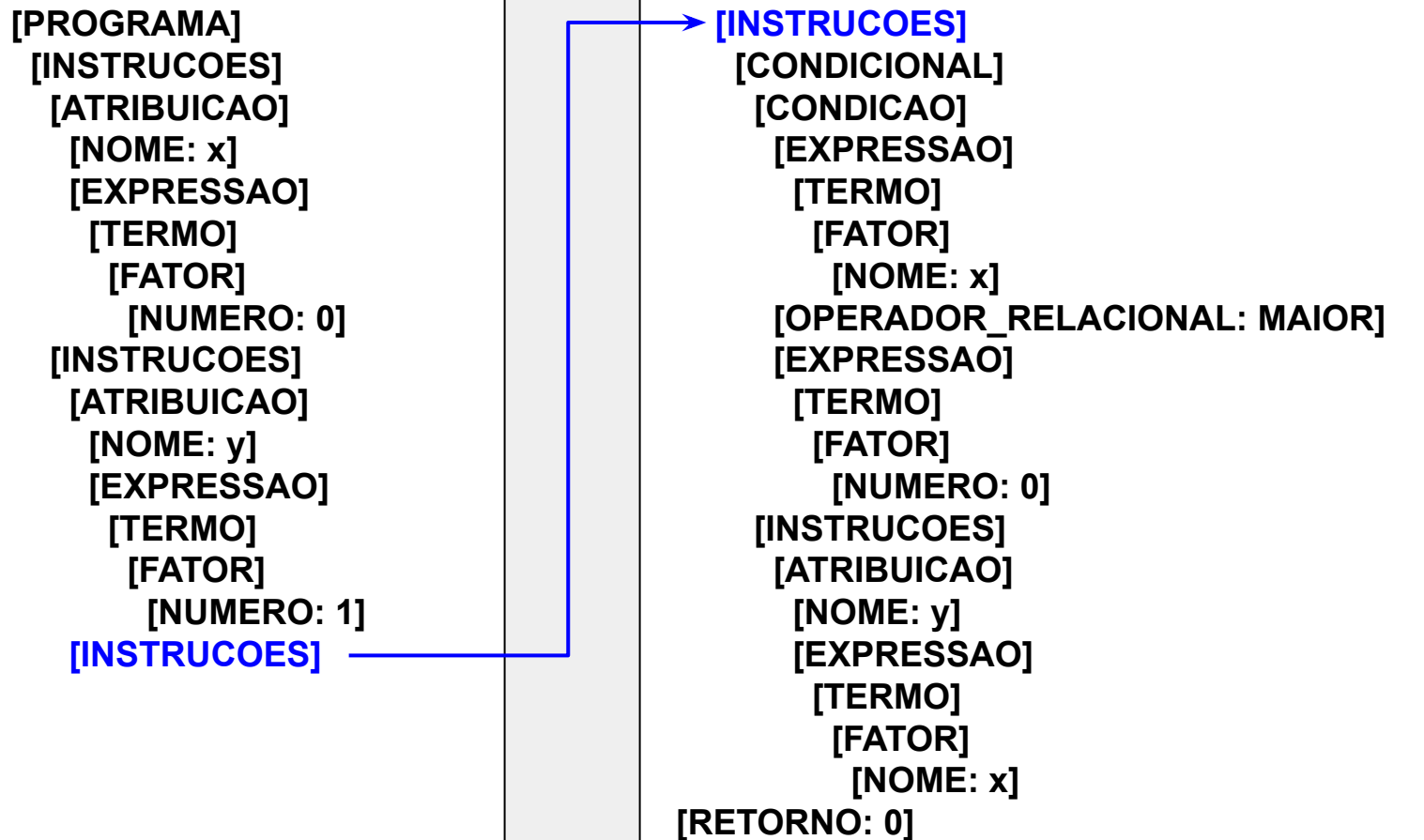
Exercício 1

Crie um programa que lê um valor e exiba seu dobro caso seja positivo, caso contrário exiba “VALOR INVÁLIDO”.

Faça o programa acima em *MineonScript*, ou na linguagem de transpilação que desenvolvestes, e então desenhe a AST resultante conforme sua BNF.

Faça a engenharia reversa e monte o código gerador desta AST

[PROGRAMA]
 [INSTRUcoes]
 [ATRIBUICAO]
 [NOME: x]
 [EXPRESSAO]
 [TERMO]
 [FATOR]
 [NUMERO: 0]
 [INSTRUcoes]
 [ATRIBUICAO]
 [NOME: y]
 [EXPRESSAO]
 [TERMO]
 [FATOR]
 [NUMERO: 1]
 [INSTRUcoes]



[INSTRUcoes]
 [CONDICIONAL]
 [CONDICAO]
 [EXPRESSAO]
 [TERMO]
 [FATOR]
 [NOME: x]
 [OPERADOR_RELACIONAL: MAIOR]
 [EXPRESSAO]
 [TERMO]
 [FATOR]
 [NUMERO: 0]
 [INSTRUcoes]
 [ATRIBUICAO]
 [NOME: y]
 [EXPRESSAO]
 [TERMO]
 [FATOR]
 [NOME: x]
 [RETORNO: 0]

TESTE



<https://docs.google.com/forms/d/e/1FAIpQLSdKkugh1TOgLIJNPBpuRDuAqE-sKkoFwrhSDiVqelrwmEPREg/viewform>

TESTE

Questão 1 O que é uma Árvore de Sintaxe Abstrata (AST) no contexto da construção de compiladores?

Uma representação em árvore hierárquica e simplificada do código-fonte, focada na estrutura lógica do programa e desprovida de detalhes sintáticos irrelevantes como parênteses, chaves e vírgulas.

Uma lista encadeada de tokens gerada diretamente pelo analisador léxico para representar a sequência linear de palavras reservadas do programa.

Um gráfico de dependências de memória utilizado no momento da execução para alocar variáveis globais e locais na pilha (stack).

Uma tabela Hash que armazena os endereços de memória, escopos e tipos de todos os identificadores declarados no arquivo.

Uma representação gráfica exata e completa do código contendo cada caractere lido do arquivo, incluindo espaços em branco, quebras de linha e comentários.

Questão 2

Qual é o papel principal do Analisador Sintático (Parser) em relação à AST?

Ler a sequência de tokens fornecida pelo analisador léxico, validar se ela respeita as regras da gramática (BNF) e construir a estrutura da AST na memória.

Percorrer o texto caractere por caractere para identificar palavras reservadas usando expressões regulares antes de criar a AST.

Traduzir os nós da AST diretamente para código de máquina executável no barramento do processador sem etapas intermediárias.

Analisar os tipos de dados das variáveis dentro da AST e emitir mensagens de erro de tipagem antes de verificar a sintaxe.

Converter o arquivo de código-fonte diretamente em uma imagem vetorial para visualização gráfica das funções.

Questão 3

Ao analisar uma expressão matemática como $2 + 3 * 4$, como a precedência dos operadores se reflete na estrutura da AST?

O operador de maior precedência (*) fica em um nível mais profundo da árvore, sendo filho da operação de menor precedência (+), garantindo que seja avaliado primeiro.

Análise Sintática: Parser

```
class Parser:
    # Método construtor para 3 atributos.
    def __init__(self, tokens):
        self.tokens = tokens
        self.pos = 0
        self.token_atual = self.tokens[self.pos]
    # Verifica o token atual e avança a ponteiro.
    def consumir(self, tipo_esperado) : (...)
    # Analisadores sintáticos
    def parse_programa(self) : (...)
    def parse_instrucoes(self) : (...)
    def parse_instrucao(self) : (...)
    def parse_atribuicao_ou_entrada(self) : (...)
    def parse_saida(self) : (...)
    def parse_condicional(self) : (...)
    def parse_loop(self) : (...)
    def parse_condicao(self) : (...)
    def parse_expressao(self) : (...)
    def parse_termo(self) : (...)
    def parse_fator(self) : (...)
    def parse_retorno(self) : (...)
```

MineonScript

```
import re # regular expressions
import sys # Sistema

# Definição dos tipos de tokens e seus padrões.
TOKENS = [
    ("COMENTARIO", r"//.*"), #onde '.' sign. qualquer caracter com exceção de '\n'.
    ("ESPACO", r"\s+"), #onde '\s' sign. ' ',\t,\n ou \r.
    ("SE", r"\bse\b"),
    ("SENAO", r"\bsenao\b"),
    ("ENQUANTO", r"\benquanto\b"),
    ("ESCREVA", r"\bescreva\b"),
    ("LEIA", r"\bleia\b"),
    ("RETORNO", r"\bretorna\b"),
    ("IGUALDADE", r"=="),
    ("IGUAL", r "="),
    ("MAIOR", r ">"),
    ("MENOR", r "<"),
    ("NEGACAO", r "!"),
    ("SOMA", r "\+"),
    ("SUB", r "-"),
    ("MUL", r "\*"),
    ("DIV", r "/"),
    ("ABRE_CHAVE", r "{"),
    ("FECHA_CHAVE", r "}"),
    ("ABRE_PAR", r "("),
    ("FECHA_PAR", r ")"),
    ("NUMERO", r "\d+"),
    ("TEXTTO", r "\".*?\\"",),
    ("NOME", r "[a-zA-Z_][a-zA-Z0-9_]*"),
```


MineonScript - Versão final com parser

```
import re # regular expressions
import sys # Sistema

# Definição dos tipos de tokens e seus padrões.
TOKENS = [
    ("COMENTARIO", r"//.*"),
    ("ESPACO", r"\s+"),
    ("SE", r"\bse\b"),
    ("SENAO", r"\bsenao\b"),
    ("ENQUANTO", r"\benquanto\b"),
    ("ESCREVA", r"\bescreva\b"),
    ("LEIA", r"\bleia\b"),
    ("RETORNO", r"\bretorna\b"),
    ("IGUALDADE", r"=="),
    ("IGUAL", r "="),
    ("MAIOR", r ">"),
    ("MENOR", r "<"),
    ("NEGACAO", r "!"),
    ("SOMA", r "+"),
    ("SUB", r "-"),
    ("MUL", r "*"),
    ("DIV", r "/"),
    ("ABRE_CHAVE", r "{"),
    ("FECHA_CHAVE", r "}"),
    ("ABRE_PAR", r "("),
    ("FECHA_PAR", r ")"),
    ("NUMERO", r "\d+"),
    ("TEXTO", r "\".*?\\"",),
    ("NOME", r "[a-zA-Z_][a-zA-Z0-9_]*"),
```